Code:

```
import math
from heapq import heappush, heappop

# ------------------------------
# Heuristic Functions
# ------------------------------

def manhattan(current, goal):
    return abs(current[0] - goal[0]) + abs(current[1] - goal[1])



def euclidean(current, goal):
    return math.sqrt(
        (current[0] - goal[0])**2 +
        (current[1] - goal[1])**2
    )



# ------------------------------
# A* Search Algorithm
# ------------------------------

def astar(grid, start, goal, heuristic):

    rows = len(grid)
    cols = len(grid[0])

    # Priority Queue: (f, g, current_node, path)
    open_list = []
```

```
heappush(open_list, (0, 0, start, [start]))

visited = set()

while open_list:

    f, g, current, path = heappop(open_list)

    # Goal reached
    if current == goal:
        return path, g

    # Skip visited nodes
    if current in visited:
        continue

    visited.add(current)

    x, y = current

    # Four possible movements
    directions = [
        (1, 0),    # Down
        (-1, 0),   # Up
        (0, 1),    # Right
        (0, -1)    # Left
    ]

    for dx, dy in directions:
```

```
            nx = x + dx
            ny = y + dy

            # Check boundary and obstacle
            if (0 <= nx < rows and
                0 <= ny < cols and
                grid[nx][ny] == 0):

                if (nx, ny) not in visited:

                    new_g = g + 1
                    h = heuristic((nx, ny), goal)
                    new_f = new_g + h

                    heappush(
                        open_list,
                        (
                            new_f,
                            new_g,
                            (nx, ny),
                            path + [(nx, ny)]
                        )
                    )

    return None, None


# ------------------------------
# Driver Program
# ------------------------------
```

```
grid = [
    [0, 0, 0, 0, 1],
    [1, 1, 0, 1, 0],
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0]
]

start = (0, 0)
goal = (4, 4)

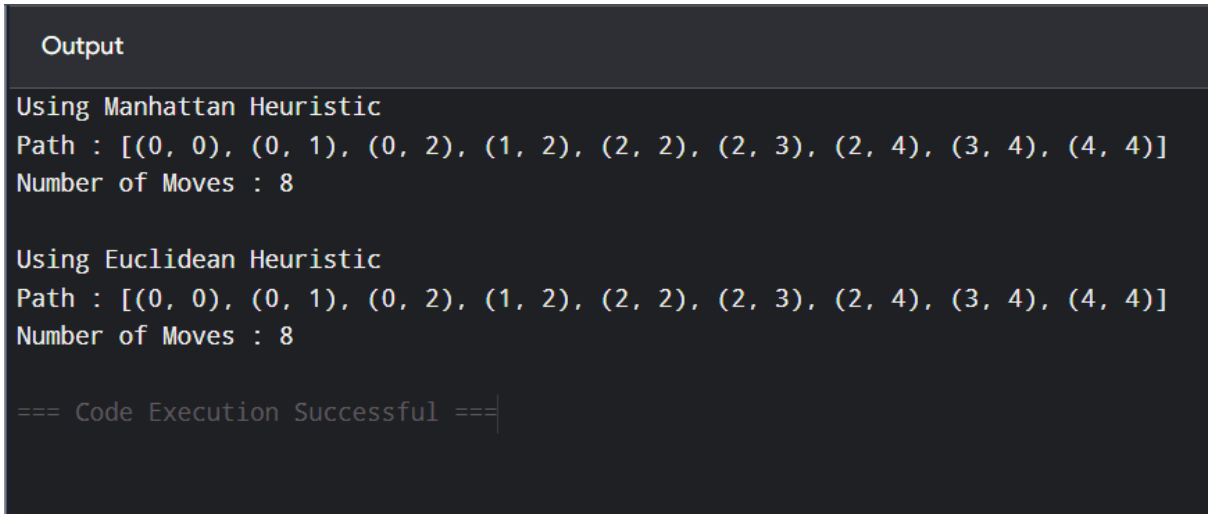

# Manhattan Heuristic
path, cost = astar(grid, start, goal, manhattan)

print("Using Manhattan Heuristic")
print("Path :", path)
print("Number of Moves :", cost)
print()


# Euclidean Heuristic
path, cost = astar(grid, start, goal, euclidean)

print("Using Euclidean Heuristic")
print("Path :", path)
print("Number of Moves :", cost)
```

OUTPUT:


Output
Using Manhattan Heuristic
Path : [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (2, 4), (3, 4), (4, 4)]
Number of Moves : 8
Using Euclidean Heuristic
Path : [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (2, 4), (3, 4), (4, 4)]
Number of Moves : 8
=== Code Execution Successful ===